

Lesson 2: If statements

The ability to control the flow of your program, letting it make decisions on what code to execute, is valuable to the programmer. The if statement allows you to control if a program enters a section of code or not based on whether a given condition is true or false. One of the important functions of the if statement is that it allows the program to select an action based upon the user's input. For example, by using an if statement to check a user entered password, your program can decide whether a user is allowed access to the program.

Without a conditional statement such as the if statement, programs would run almost the exact same way every time. If statements allow the flow of the program to be changed, and so they allow algorithms and more interesting code.

Before discussing the actual structure of the if statement, let us examine the meaning of TRUE and FALSE in computer terminology. A true statement is one that evaluates to a nonzero number. A false statement evaluates to zero. When you perform comparison with the relational operators, the operator will return 1 if the comparison is true, or 0 if the comparison is false. For example, the check `0 == 2` evaluates to 0. The check `2 == 2` evaluates to a 1. If this confuses you, try to use a cout statement to output the result of those various comparisons (for example `cout<< ( 2 == 1 );`)

When programming, the aim of the program will often require the checking of one value stored by a variable against another value to determine whether one is larger, smaller, or equal to the other.

There are a number of operators that allow these checks.

Here are the relational operators, as they are known, along with examples:

>	greater than	5 > 4 is TRUE
<	less than	4 < 5 is TRUE
>=	greater than or equal	4 >= 4 is TRUE
<=	less than or equal	3 <= 4 is TRUE
==	equal to	5 == 5 is TRUE
!=	not equal to	5 != 4 is TRUE

It is highly probable that you have seen these before, probably with slightly different symbols. They should not present any hindrance to understanding. Now that you understand TRUE and FALSE in computer terminology as well as the comparison operators, let us look at the actual structure of if statements.

The structure of an if statement is as follows:

```
if ( TRUE )  
    Execute the next statement
```

To have more than one statement execute after an if statement that evaluates to true, use braces, like we did with the body of a function. Anything inside braces is called a compound statement, or a block.

For example:

```
if ( TRUE ) {  
    Execute all statements inside the braces  
}
```

There is also the else statement. The code after it (whether a single line or code between brackets) is executed if the if statement is FALSE.

It can look like this:

```
if ( TRUE ) {  
    // Execute these statements if TRUE  
}  
else {  
    // Execute these statements if FALSE  
}
```

One use for else is if there are two conditional statements that may both evaluate to true, yet you wish only one of the two to have the code block following it to be executed. You can use an else if after the if statement; that way, if the first statement is true, the else if will be ignored, but if the if statement is false, it will then check the condition for the else if statement. If the if statement was true the else statement will not be checked. It is possible to use numerous else if statements.

Let's look at a simple program for you to try out on your own.

```
#include <iostream>  
  
using namespace std;  
  
int main()  
{  
    int age; // Most important part of the program!  
    // Need a variable...  
  
    cout<<"Please input your age: "; // Asks for age  
    cin>> age; // The input is put in age  
    cin.ignore(); // Throw away enter  
    if ( age < 100 ) { // If the age is less than 100  
        cout<<"You are pretty young!\n"; // Just to show you it works...  
    }  
    else if ( age == 100 ) { // I use else just to show an example  
        cout<<"You are old\n"; // Just to show you it works...  
    }  
    else {  
        cout<<"You are really old\n"; // Executed if no other statement is  
    }  
    cin.get();  
}
```

Boolean operators allow you to create more complex conditional statements. For example, if you wish to check if a variable is both greater than five and less than ten, you could use the boolean AND to ensure both `var > 5` and `var < 10` are true. In the following discussion of boolean operators, I will capitalize the boolean operators in order to distinguish them from normal english. The actual C++ operators of equivalent function will be described further into the tutorial - the C++ symbols are not: OR, AND, NOT, although they are of equivalent function.

When using if statements, you will often wish to check multiple different conditions. You must understand the Boolean operators OR, NOT, and AND. The boolean operators function in a similar way to the comparison operators: each returns 0 if evaluates to FALSE or 1 if it evaluates to TRUE.

NOT: The NOT operator accepts one input. If that input is TRUE, it returns FALSE, and if that input is FALSE, it returns TRUE. For example, NOT (1) evaluates to 0, and NOT (0) evaluates to 1. NOT (any number but zero) evaluates to 0. In C and C++ NOT is written as !. NOT is evaluated prior to both AND and OR.

AND: This is another important command. AND returns TRUE if both inputs are TRUE (if 'this' AND 'that' are true). (1) AND (0) would evaluate to zero because one of the inputs is false (both must be TRUE for it to evaluate to TRUE). (1) AND (1) evaluates to 1. (any number but 0) AND (0) evaluates to 0. The AND operator is written && in C++. Do not be confused by thinking it checks equality between numbers: it does not. Keep in mind that the AND operator is evaluated before the OR operator.

OR: Very useful is the OR statement! If either (or both) of the two values it checks are TRUE then it returns TRUE. For example, (1) OR (0) evaluates to 1. (0) OR (0) evaluates to 0. The OR is written as || in C++. Those are the pipe characters. On your keyboard, they may look like a stretched colon. On my computer the pipe shares its key with \. Keep in mind that OR will be evaluated after AND.

It is possible to combine several boolean operators in a single statement; often you will find doing so to be of great value when creating complex expressions for if statements. What is !(1 && 0)? Of course, it would be TRUE. It is true is because 1 && 0 evaluates to 0 and !0 evaluates to TRUE (ie, 1).

Try some of these - they're not too hard. If you have questions about them, feel free to stop by our forums.

- A. !(1 || 0) ANSWER: 0
B. !(1 || 1 && 0) ANSWER: 0 (AND is evaluated before OR)
C. !((1 || 0) && 0) ANSWER: 1 (Parenthesis are useful)

If you find you enjoyed this section, then you might want to look more at Boolean Algebra.